Big Data for Autonomous Vehicles

Venkata Rao Gaddam
Student ID: 14513842
Email: gaddamv4@coventry.ac.uk

Table of Contents

Table of Figures

Table of Tables

1 Preface

1.1 About this document

The rapid advancements in autonomous vehicle technology have ushered in a new era of intelligent transportation systems, where the integration of Big Data plays a crucial role. Autonomous vehicles rely on vast amounts of data collected from diverse sources such as sensors, cameras, LiDAR, RADAR, GPS, and communication systems to perceive and interact with their environment. The ability to process, analyze, and leverage this data in real time is critical to ensuring the safety, efficiency, and reliability of autonomous systems.

This document elicits the foundational role of Big Data in the development of autonomous vehicles. It delves into how advanced analytics, Machine learning (ML) and data driven decision-making enable key functionalities such as obstacle detection, traffic-sign recognition, predictive maintenance, and route optimization. Additionally, it highlights the challenges associated with managing, storing, processing such massive datasets and emphasizes the need for innovative tools and techniques to address these complexities in leveraging Big Data for autonomous vehicles.

1.2 Document Control

Status:	Release
Version:	1.0
Security Classification:	University Confidential
Author(s):	Venkata Rao Gaddam
Issue Date:	18-Dec-2024

1.3 Revision History

Date	Version	Author	Comments
9-Dec-2024	0.1	Venkata Rao G	Initial document
18-Dec-2024	1.0	Venkata Rao G	Review & update

1.4 References

No.	Document	Reference	Revision
1	Course Work Assessment	Assessment_Brief_CW.pdf	NA

2	Deep Learning and Big Data for ITS Book	DeepLearning_BigData_Book.pdf	NA
3	Big Data Applications	https://medium.com/@aakash.bachheriya123/will-real-time-big-data-applications-ever-rule-the-world-323b88656b1e	NA
4	The Role of Big Data in AVs	https://www.researchgate.net/publication/359942241_The_Role_of_Big_Data_and_Smart_Technologies_in_Autonomous_Vehicles	NA
5	Big Data for Autonomous Vehicles	https://www.researchgate.net/publication/350795005_Big_Data_for_Autonomous_Vehicles	NA
6	CNN based Traffic Sign Recognition Paper	https://www.researchgate.net/publication/368500246_CNN-Based_Traffic_Sign_Recognition	NA
7	GTSRB IEEE Paper	https://ieeexplore.ieee.org/document/6033395	NA
8	Google Colab	https://colab.research.google.com/	NA
9	Kaggle Datasets	https://www.kaggle.com/datasets?search=GTSRB	NA
10	GTSRB Dataset	https://benchmark.ini.rub.de/gtsrb_dataset.html	NA

1.5 Terms and Abbreviations

Abbreviation	Meaning
AD	Autonomous Driving
ADAS	Advanced Driver Assistance Systems
AEV	Autonomous Electrical Vehicle
AI	Artificial Intelligence
AV	Autonomous Vehicle
BT	Bluetooth
CAN	Controller Area Network
CAN-FD	CAN Flexible Datarate
DL	Deep Learning
CNN	Convolutional Neural Network
DSRC	Dedicated Short-Range Communication
DUT	Device Under Test
ECU	Electronic Control Unit
GPS	Global Positioning System

Abbreviation	Meaning
GNSS	Global Navigation Satellite System
HOG	Histogram of Oriented Gradients
IDS	Intrusion Detection Systems
ITS	Intelligent Transportation Systems
IVI	In-Vehicle Infotainment
IVN	In-Vehicle Network
KNN	K-Nearest Neighbor
LAN	Local Area Network
LIN	Local Interconnect Network
LiDAR	Light Detection and Ranging
ML	Machine Learning
OBD-II	On-board Diagnostics Port
OEM	Original Equipment Manufacturer
OTA	Over-The-Air
PCA	Principal Component Analysis
RADAR	Radio Detection and Ranging
RNN	Recurrent Neural Network
SAE	Society of Automotive Engineers
SUV	Sport Utility Vehicle
USB	Universal Serial Bus
UK	United Kingdom
V2X	Vehicle-to-Everything
Wi-Fi	Wireless Fidelity

2 Executive Summary

The rise of autonomous vehicles (AV) represents a transformative shift with Big Data in Intelligent Transportation Systems (ITS), serving as the cornerstone of their development and operation by processing vast amounts of data from sensors, cameras, LiDAR, Radar, GPS, inter and intra vehicle communication networks. Big Data for autonomous vehicle enables the real-time decision-making required for safe, efficient, and intelligent vehicle functionality. By enabling real-time analytics, predictive modelling, and intelligent decision-making and it is paving the way for a future of connected, smart, and sustainable transportation systems.

This report explores the role of Big Data in autonomous vehicles, highlighting its applications, challenges, management & interface tools, associated ML algorithms and future potential[1].

2.1 Big Data Introduction

Big Data refers to extremely large and complex datasets that are difficult to process, analyze, and store using traditional data management systems. The rapid digitization of various sectors, the proliferation of connected devices, and the rise of the Internet of Things (IoT) have all contributed the generation of unprecedented amounts of data. Big Data has become an essential asset in diverse domains, including healthcare, finance, retail, and the automotive industry, enabling data-driven decision-making and innovation[2].

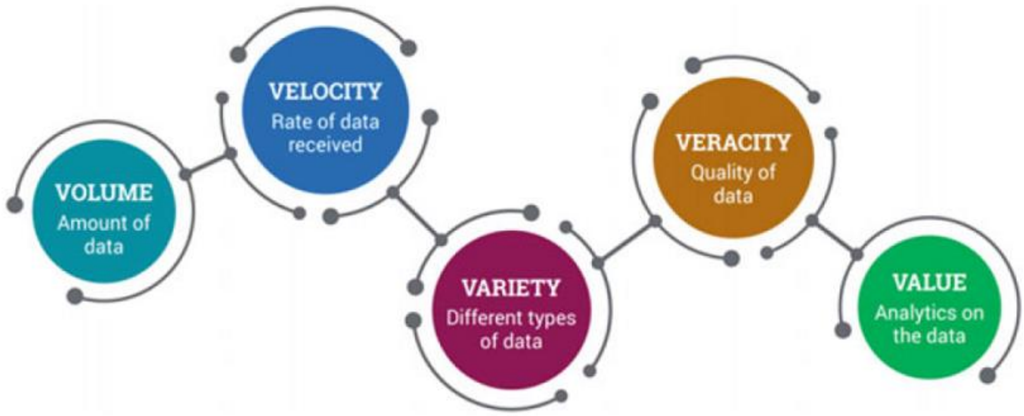


Figure 1: Characteristics of Big Data[2]

Big Data is commonly described using the "5Vs" framework:

1. **Volume**: The sheer size of data generated. For instance, connected vehicles alone can produce terabytes of data per day.
2. **Velocity**: The speed at which data is generated and needs to be processed (e.g., real-time analytics for autonomous driving).
3. **Variety**: Different types of data, such as structured (databases, excel sheets), semi-structured (XML, JSON), and unstructured (images, videos, sensor data, docs, social media posts).
4. **Veracity**: The reliability and accuracy of the data, which is critical for meaningful insights.
5. **Value**: The potential economic and strategic benefits derived from analyzing the data.

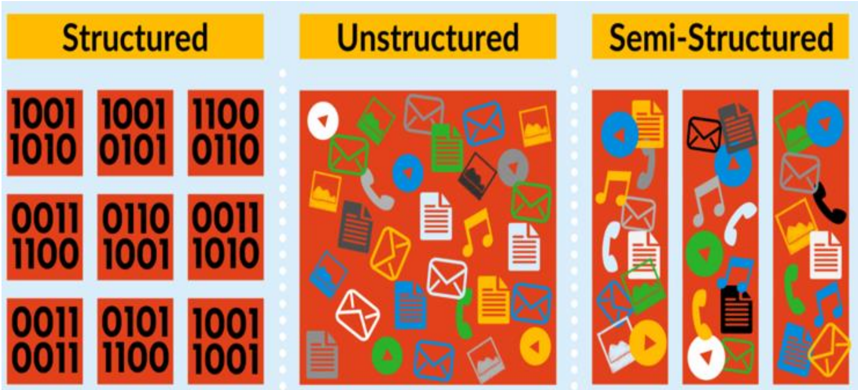


Figure 2: Types of Big Data

Now a days, Big Data is very crucial for organizations aiming to stay competitive in an increasingly digital world. Its importance lies in the ability to:

- **Enhance Decision-Making**: Through advanced analytics, businesses can derive actionable insights to improve operations and strategy.
- **Predict Trends**: Predictive analytics leverages historical data to forecast future trends.
- **Drive Innovation**: Big Data fuels innovation in product design, service delivery, and process improvements.
- **Enable Personalization**: Customizing user experiences based on individual preferences and behaviors.

2.2 Key Components of Big Data

1. **Data Sources**: Big Data is generated from multiple sources, including:
   - IoT Devices: Sensors, smart appliances, and connected vehicles.
   - Social Media: Platforms like Twitter, Facebook, and Instagram.
   - Enterprise Data: CRM, ERP, and transaction systems.
   - Public Data: Open datasets from governments and research institutions.
   - Search Engine Data: Search engines retrieve lots of data from different databases.
   - Stock Exchange Data: The stock exchange data holds information about the 'buy' and 'sell' decisions made on a share of different companies made by the customers.
2. **Data Storage**: Big Data requires scalable storage solutions to manage large datasets such as: Hadoop Distributed File System (HDFS) for unstructured data and Cloud Storage Platforms like AWS S3 and Google Cloud Storage.
3. **Data Processing**: involves data acquisition, data exploration, data pre-processing and data analysis.

Process Steps	Description	Techniques	Tools & Technologies
Data Acquisition	Process of collecting, measuring, and recording data from various sources for analysis, monitoring, control, or storage.		Apache Kafka, Apache Nifi
Exploring Data	Process of examining and analyzing data to understand its main characteristics, patterns, and relationships.	a) **Descriptive statistics**: Measures of central tendency (mean, median, mode), Measures of dispersion (range, variance, standard deviation), Percentiles and quartiles. b) **Data visualization**: Histograms and box plots for distribution analysis, Scatter plots for relationship exploration, Heat maps for correlation analysis, Time series plots for temporal data. c) **Dimensionality reduction**: Principal Component Analysis (PCA), t-SNE (t-Distributed Stochastic Neighbor Embedding). d) **Correlation analysis**: Pearson correlation for linear relationships Spearman correlation for monotonic relationships.	a) Python libraries: Pandas, NumPy, Matplotlib, Seaborn b) HADOOP AND SPARK c) R and its visualization packages d) Tableau for interactive visualizations e) Jupyter Notebooks for combining code and visualizations.
Pre-processing	Data preprocessing is the process of transforming raw data into a format that is more suitable for analysis, machine learning, or other data science tasks. Common preprocessing steps: **Data profiling**: Examining and analyzing data to collect quality statistics. **Data cleansing**: Handling missing values, removing outliers, correcting errors. **Data reduction**: Reducing dimensionality or selecting relevant features. **Data transformation**: Scaling, normalization, encoding categorical variables. **Handling imbalanced data**: Oversampling, undersampling, or hybrid methods.	**Feature scaling**: Min-Max scaling, standardization, robust scaling. **Encoding categorical data**: One-hot encoding, label encoding **Dimensionality reduction**: Principal Component Analysis (PCA). **Feature selection**: Correlation analysis, statistical tests. **Data discretization**: Converting continuous data to discrete values.	Apache Spark, Hadoop MapReduce
Data Analysis	Process of applying statistical and machine learning techniques to extract insights from the prepared data. This stage may involve predictive modeling, clustering, and other advanced analytics methods to uncover valuable information and support decision-making.		

Table 1: Data Processing Steps in Big Data

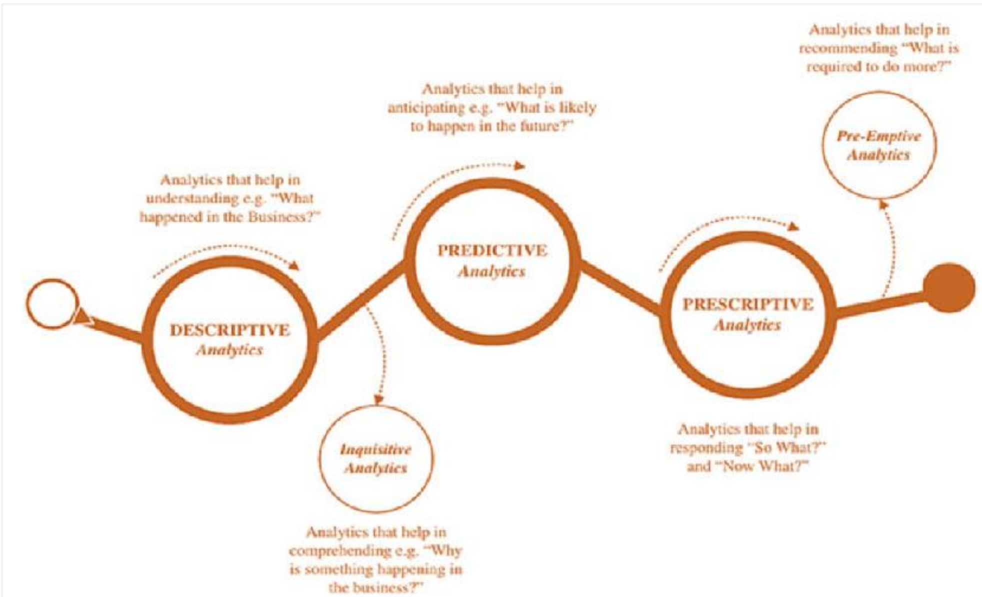


Figure 3: Analytical Methods of Big Data[2]

2.3 Applications of Big Data

Big Data has transformative applications across industries and has been applied across diverse domains, enabling new opportunities and addressing complex challenges.

Below is a critical evaluation of its major applications:

Industry	Applications	Challenges
Healthcare	a) Disease prediction and personalized treatment through patient data analysis. b) Drug discovery using genomic data and clinical trial analytics. c) Real-time monitoring of patients through wearable devices.	a) **Data privacy**: Sensitive patient information is prone to breaches. b) **Interoperability**: Diverse healthcare systems often lack standardization, complicating data integration. c) **Bias in data**: Models trained on incomplete or biased datasets can lead to incorrect diagnoses.
Automotive	a) **Autonomous vehicles**: Big Data enables sensor fusion and decision-making. b) **Predictive maintenance**: Analyzing sensor data to forecast component failures. c) **Fleet management:** Optimizing routes and logistics using real-time analytics. d) **Manufacturing optimization:** ML-powered anomaly detection ensures quality control and minimizes downtime.	a) **Real-time processing**: High latency in processing data can compromise safety. b) **Data volume**: Autonomous vehicles generate terabytes of data daily, creating storage and processing challenges.
Retail and E-commerce	a) Customer behavior analysis for personalized marketing. b) Dynamic pricing based on real-time demand and competitor strategies. c) Inventory management through predictive analytics.	**Data silos**: Fragmented datasets across platforms hinder comprehensive analysis. **Over-reliance on predictions**: Inaccurate forecasts can lead to customer dissatisfaction or financial loss.
Finance	a) Fraud detection using pattern recognition and anomaly detection. b) Algorithmic trading based on real-time market data. c) Credit scoring through customer behavior analysis.	a) **Model transparency**: ML models often function as black boxes, raising concerns about accountability. b) **Data quality**: Inaccurate or incomplete data can lead to incorrect decisions.
Energy and Utilities	a) Smart grid management for optimized energy distribution. b) Predictive maintenance of equipment in power plants. c) Renewable energy optimization through weather data analysis.	a) **Scalability:** Managing data from millions of IoT-enabled devices. b) **Cybersecurity**: Critical infrastructure is vulnerable to cyberattacks.
Manufacturing	Predictive maintenance by analyzing equipment sensor data for potential failures and schedule maintenance activities. It reduces downtime but also extends the lifespan of critical assets.	a) **Data volume**: large volumes of data from the manufacturing industry are untapped. 2) **Data usage**: underutilization of this information prevents the improved quality of products, energy efficiency, reliability, and better profit margins.
Education	Helps in understanding student performance, engagement, and learning patterns. Institutions can make data-informed decisions to improve curriculum design and student success.	a) **Interoperability**: integrating data from different sources on different platforms and from different vendors that were not designed to work with one another. b) **Data privacy**: Issues of privacy and personal data protection.

Table 2: Major Application of Big Data

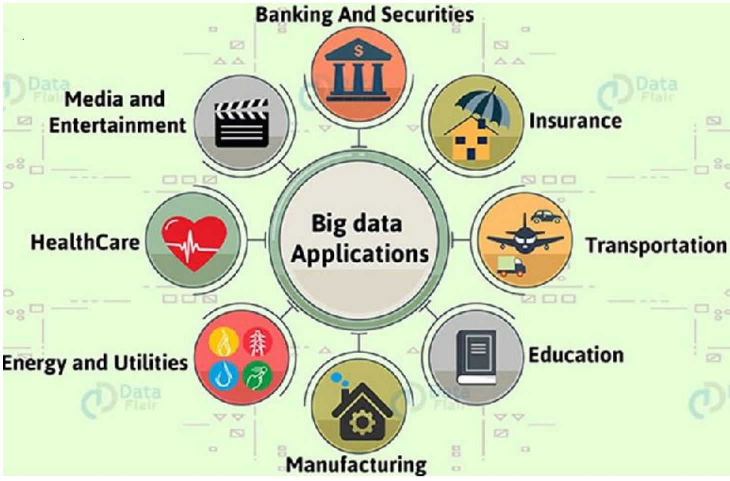


Figure 4: Top Applications of Big Data[3]

2.4 Challenges in Big Data

Despite its advantages, Big Data poses challenges:

- **Data Security and Privacy**: Safeguarding sensitive data from breaches.
- **Data Quality**: Ensuring accurate and clean datasets.
- **Integration**: Combining data from multiple disparate sources.
- **Scalability**: Managing the ever-increasing volume and complexity of data.

2.5 Future of Big Data

Big Data is evolving, with emerging trends shaping its future:

- **Edge Computing**: Processing data closer to its source to reduce latency.
- **AI Integration**: Leveraging Artificial Intelligence for advanced analytics.
- **Quantum Computing**: Potentially revolutionizing Big Data analytics with unprecedented processing power.
- **Sustainability**: Using Big Data to optimize energy consumption and promote green initiatives.

3 Machine Learning Algorithms in Big Data

Big data involves extremely large datasets that high in volume, velocity, and variety. Machine Learning (ML) algorithms designed for big data and are essential for analyzing Big Data, particularly in solving classification and clustering problems using specialized frameworks like Hadoop, Apache Spark, or TensorFlow to manage the computational load. Together, they are driving innovations in data analytics and artificial intelligence across industries.

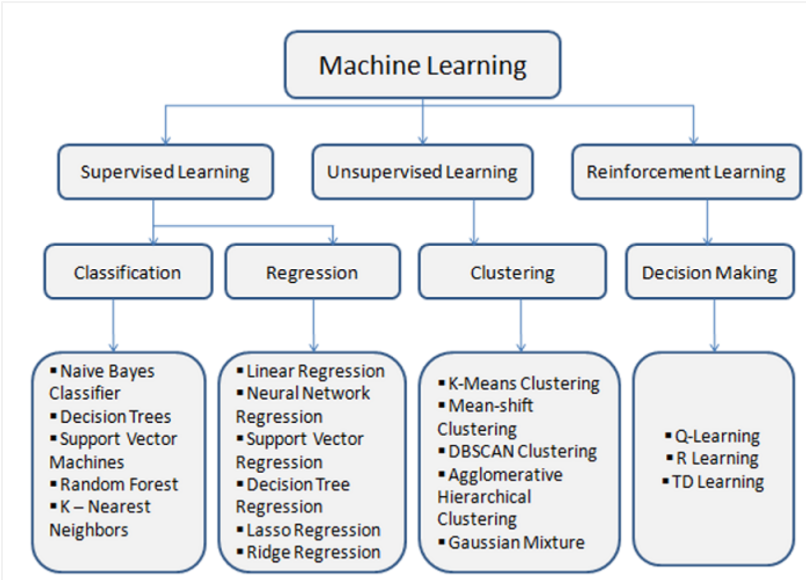


Figure 5: Machine Learning Algorithms

3.1 ML Algorithms for Classification

Classification involves assigning labels to high-dimensional and diverse datasets and categorizing data points into predefined classes based on their attributes. Some scalable algorithms include:

- **Logistic Regression**: Handles binary classification by predicting probabilities. Frameworks like Spark MLlib scale logistic regression to large datasets.
- **Decision Trees**: Builds a tree-like model to make decisions based on data attributes or feature splits.
- **Random Forests**: Ensemble learning method that combines multiple decision trees to improve accuracy. Parallel computation in frameworks like Hadoop enables scalability.
- **Support Vector Machines (SVM)**: Finds the hyperplane that best separates classes with clear margins in a dataset.
- **Deep Learning Models (CNN, RNN or KNN)**: For large-scale image or sequential data classification, distributed frameworks like TensorFlow and PyTorch are utilized.

3.2 ML Algorithms for Clustering

Clustering involves grouping data points based on their similarities without predefined labels. In big data, algorithms need to handle large-scale computations effectively.

- **K-Means Clustering**: Partitions data into k clusters by minimizing intra-cluster variance. Optimized for large datasets by parallelizing computation and using map-reduce techniques (in Spark MLlib).

- **Hierarchical Clustering**: Builds a hierarchy of clusters using agglomerative or divisive approaches.
- **DBSCAN (Density-Based Spatial Clustering of Applications with Noise)**: Groups data points based on density, identifying noise as outliers.
- **Gaussian Mixture Models (GMM)**: Uses probabilistic models to represent clusters.
- **Spectral Clustering**: Efficient implementations approximate eigenvector calculations to make spectral clustering viable for big data.

3.3 Challenges and Techniques in Big Data ML

- **High Dimensionality**: Use of dimensionality reduction techniques like PCA or feature selection. Algorithms like Random Forest inherently reduce dimensionality by focusing on feature importance.
- **Data Distribution**: Algorithms leverage distributed frameworks like Apache Spark or TensorFlow to parallelize computations. For instance, K-Means runs iterative computations across multiple nodes in Spark.
- **Scalability**: Algorithms are optimized for batch processing and iterative refinement.
- **Imbalanced Data**: Classification models use techniques like oversampling, undersampling, or weighted loss functions to handle skewed datasets.

3.4 Solving Classification and Clustering Problems

Classification Problems	a) **Feature Learning**: Extracting the most relevant information from the input data.
	b) **Boundary Identification**: Algorithms like SVMs or Neural Networks find decision boundaries to separate classes effectively.
	c) **Generalization**: Training models on labeled data enables them to generalize to unseen data.
Clustering Problems	a) **Identifying Natural Groupings**: Finding clusters based on similarity metrics like Euclidean distance or density.
	b) **Discovering Patterns**: Revealing hidden relationships, such as customer segmentation in marketing.
	c) **Reducing Complexity**: Summarizing large datasets into representative groups.

Table 3: Solutions for Classification and Clustering Problems

4 Building ML Models for Autonomous Vehicles, Traffic Sign Recognition

This section talks about building analytical and predictive machine learning models to classify traffic signs using open dataset, German Traffic Sign Recognition Benchmark (GTSRB) for Autonomous Vehicles.

The road sign recognition is one of the most important tasks for autonomous vehicles to drive safely and avoid road accidents. This efficiently relies on their ability to accurately recognize the traffic signs and makes traffic sign recognition is a crucial component. The process of traffic sign recognition involves several key steps, including data preprocessing, feature extraction, classification and clustering.

The dataset preparation for Traffic Sign Recognition (TSR) involves Data Collection (use public datasets or own datasets), Data Annotation (labeling the images), Data Preprocessing (image resizing, normalization, augmentation), Splitting the dataset (training, validation and testing), Data format and balancing & evaluate the dataset.

This report uses Jupyter Notebook to develop the code with five different ML Models for an automotive-related classification (supervised) and clustering (unsupervised) tasks. These models analyze, train and evaluate the model's accuracy and performance.

1. Random Forest trains multiple decision trees and aggregates predictions, effective for moderately complex datasets.
2. KNN uses raw pixel data or extracted features, simple but expensive for large datasets.
3. Transfer learning to leverage a pre-trained model to improve efficiency and accuracy.
4. CNN by keeping the model size relatively small but enough to achieve comparable or better performance than with transfer learning.
5. K-Means Clustering, group similar traffic signs (Speed limits, Warnings, etc.) based on pixel-level similarities.

4.1 Traffic Sign Recognition Dataset Overview

The traffic sign recognition is a multi-category classification task, involves two steps, detection of the traffic sign and recognition after classification & clustering using machine learning models. The quality of dataset plays a major role here, therefore selected GTSRB dataset for this assignment This dataset is ideal for building analytical and predictive models for traffic-sign recognition, an essential component of autonomous driving systems.

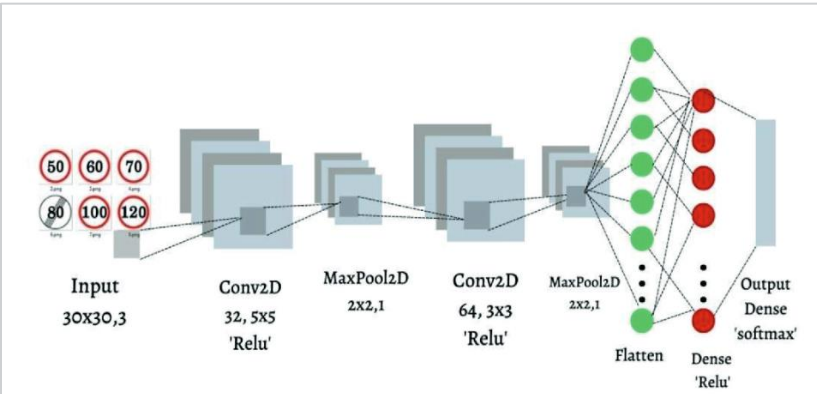


Figure 6: Sequence of steps to classify Traffic Image Signs

The most important open datasets for traffic sign are shown in Figure 7.

	GTSRB	STS Dataset	KUL Dataset	RUG Dataset	Stereopolis	LISA Dataset
Number of classes:	43	7	100+	3	10	49
Number of annotations:	50000+	3488	13444	0	251	7855
Number of images:	50000+	20000	9006	48	847	6610
Annotated images:	All images	4000 images	All images	0	All images	All images
Sign sizes:	15x15 to 250x250 px	3x5 to 263x248 px	100x100 to 1628x1236 px	N/A	25x25 to 204x159 px	6x6 to 167x168 px
Image sizes:	15x15 to 250x250 px	1280x960 px	1628x1236 px	360x270 px	1920x1080 px	640x480 to 1024x522 px
Includes videos:	No	No	Yes, 4 tracks	No	No	Yes, for all annotations
Country of origin:	Germany	Sweden	Belgium	The Netherlands	France	United States
Extra info:	Images come in tracks with 30 different images of the same physical sign.	Signs marked visi-ble/blurred/occluded and whether they belong to the current road or a side road.	Includes traffic sign annotations, camera calibrations and poses.	Does not include any annotations, only raw pictures.		Images from various camera types.

Figure 7: Open Datasets for Traffic Sign Recognition[6]

GTSRB dataset is downloaded from the Kaggle[9], and folder structure as shown below.

```
+- -  GTSRB
¦  +- -  Meta [43 entries]
¦  +- -  Test [12631 entries]
¦  +- -  Train [43 entries]
¦  +- -  meta-1 [43 entries]
¦  +- -  test-1 [12631 entries]
¦  +- -  train-1 [43 entries]
¦  +- -  Meta.csv
¦  +- -  Test.csv
¦  +- -  Train.csv
```

The GTSRB dataset contains over 50,000 labeled images of traffic signs across 43 classes for image classification[10]. This dataset includes a wide variety of real-world conditions such as lighting variations, different perspectives, and partially occluded signs which helps in real time to recognize the image of road signs like speed limit, danger, and turn ahead signs as shown in below Figure 8.

Figure 8: GTSRB dataset for Traffic Sign Recognition[7]

Using GTSRB dataset with ML Models focuses on teaching a self-driving car to take actions based on its understanding of traffic signs which help in decisions making tasks such as navigating traffic, avoiding collisions or obeying traffic rules[7].

4.2 Implementation of ML Models

Step 1: Setup and Dataset Preparation

a) Install Required Packages

```
pip install tensorflow scikit-learn matplotlib seaborn pandas numpy opencv-python
```

b) Load and Preprocess the GTSRB Dataset

```
import tensorflow as tf
print(tf.__version__)
import keras
print(keras.__version__)

import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import seaborn as sns

from PIL import Image
from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report, confusion_matrix, accuracy_score
from keras import layers, models, applications
from keras.utils import to_categorical, plot_model
from keras.models import Sequential
from keras.layers import Dense, Flatten, Conv2D, MaxPool2D, Dropout
from keras.applications import ResNet50

import os

from google.colab import drive # Import the drive library

# Mount Google Drive
drive.mount('/content/drive') # Mount to access files
# Define the path to the GTSRB dataset
dataset_path = '/content/drive/MyDrive/Colab Notebooks/Dataset/Train'
#dataset_path = '/content/GTSRB/Train'

print("Num GPUs Available: ", len(tf.config.list_physical_devices('GPU')))
```

```
data = []
labels = []
classes = 43

for i in range(classes):
  path = os. path.join(dataset_path, str(i))
  images = os.listdir(path)
  for a in images:
    try:
        image = Image.open(path + '/' + a) # Changed '\' to '/' for path separator
        image = image.resize((32,32))
        image = np.array(image)
        data.append(image)
        labels.append(i)
    except Exception as e: # Added Exception handling and print error message
        print(f"Error loading image: {a}, Error: {e}")
data = np.array(data)
labels = np.array(labels)

print(data.shape, labels.shape)
# Preprocessing data - Normilizing and resize
data = data / 255.0
#labels = labels / 255.0

#Splitting training and testing dataset
X_t1, X_t2, y_t1, y_t2 = train_test_split(data, labels, test_size=0.2, random_state=42)
print(X_t1.shape, X_t2.shape, y_t1.shape, y_t2.shape)

#Converting labels into one hot encoding
num_classes = len(np.unique(y_t1))
y_t1 = to_categorical(y_t1, num_classes)
y_t2 = to_categorical(y_t2, num_classes)
print(num_classes)
```

Step 2: Data Augmentation with adding random flips and rotations

```
# Data Augmentation Layer Integration into Training Process

# Data Augmentation with adding random flips and rotations
data_augmentation = tf.keras.Sequential([
    layers.RandomFlip("horizontal_and_vertical"),
    layers.RandomRotation(0.2),
])

# Apply data augmentation to the training dataset only
augmented_images = []
for image in X_t1:
  augmented_images.append(data_augmentation(image, training=True))
```

Step 3: Model Implementations

a) Model 1: Random Forest

```
# Random Forest

from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score

# Flatten the images
X_train_flat = X_t1.reshape(X_t1.shape[0], -1)
X_test_flat = X_t2.reshape(X_t2.shape[0], -1)

#Train Random Forest
rf_model = RandomForestClassifier(n_estimators=100, random_state=42)
rf_model.fit(X_train_flat, np.argmax(y_t1, axis=1))

# Predict
y_pred_rf = rf_model.predict(X_test_flat)

# Calculate accuracy
accuracy = accuracy_score(np.argmax(y_t2, axis=1), y_pred_rf)
print(f"Random Forest Accuracy: {accuracy}")
```

b) Model 2: K-Nearest Neighbors (KNN)

```
# K-Nearest Neighbors (KNN)
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score

# Train KNN Model
knn_model = KNeighborsClassifier(n_neighbors=5)
knn_model = knn_model.fit(X_train_flat, np.argmax(y_t1, axis=1))

#Predict
y_pred_knn = knn_model.predict(X_test_flat)

# Calculate accuracy
accuracy = accuracy_score(np.argmax(y_t2, axis=1), y_pred_knn)
print(f"KNN Accuracy: {accuracy}")
```

c) Model 3: Transfer Learning

```
# Transfer Learning with MobileNetV2
from tensorflow.keras.applications import MobileNetV2
from tensorflow.keras.models import Model

# Resize images for MobileNetV2
X_train_resized = np.array([tf.image.resize(image, (64, 64)) for image in X_t1])
X_test_resized = np.array([tf.image.resize(image, (64, 64)) for image in X_t2])

# Load Pre-trained MobileNetV2
base_model = MobileNetV2(weights='imagenet', include_top=False, input_shape=(64, 64, 3))

# Add custom classification layers
x = Flatten()(base_model.output)
x = Dense(128, activation='relu')(x)
output = Dense(num_classes, activation='softmax')(x)

mobilenet_model = Model(inputs=base_model.input, outputs=output)

# Freeze the base model layers
for layer in base_model.layers:
    layer.trainable = False

# Compile the model
mobilenet_model.compile(optimizer='adam', loss='categorical_crossentropy', metrics=['accuracy'])
```

```
# Train the model
history = mobilenet_model.fit(X_train_resized, y_t1, batch_size=32, epochs=10, validation_data=(X_test_resized, y_t2))
# predict
pred_mobilenet =  mobilenet_model.predict(X_test_resized)
y_pred_mobilenet = np.argmax(pred_mobilenet, axis=-1)

# Plot training & validation accuracy values for transfer learning
plt.figure(figsize=(10, 6))
plt.plot(history.history['accuracy'])
plt.plot(history.history['val_accuracy'])
plt.title('Transfer Learning Model with Augmentation Accuracy')
plt.ylabel('Accuracy')
plt.xlabel('Epoch')
plt.legend(['Train', 'Validation'], loc='upper left')
plt.show()

plt.figure(figsize=(10, 6))
plt.plot(history.history['loss'], label='training loss')
plt.plot(history.history['val_loss'], label='val loss')
plt.title('Transfer Learning Model with Augmentation Loss')
plt.xlabel('Epochs')
plt.ylabel('Loss')
plt.legend()
plt.show()
mobilenet_model.save('Transfer_Learning.keras')
```

d) Model 4: Convolutional Neural Network (CNN).

```
#Building the CNN model
cnn_model = Sequential()
cnn_model.add(Conv2D(filters=32, kernel_size=(5,5), activation='relu', input_shape=X_t1.shape[1:]))
cnn_model.add(Conv2D(filters=32, kernel_size=(5,5), activation='relu'))
cnn_model.add(MaxPool2D(pool_size=(2, 2)))
cnn_model.add(Dropout(rate=0.25))
cnn_model.add(Conv2D(filters=64, kernel_size=(3, 3), activation='relu'))
cnn_model.add(Conv2D(filters=64, kernel_size=(3, 3), activation='relu'))
cnn_model.add(MaxPool2D(pool_size=(2, 2)))
cnn_model.add(Dropout(rate=0.25))
cnn_model.add(Flatten())
```

```
cnn_model.add(Dense(256, activation='relu'))
cnn_model.add(Dropout(rate=0.25))
cnn_model.add(Dense(num_classes, activation='softmax')) #Compilation of the model
cnn_model.compile(loss='categorical_crossentropy', optimizer='adam', metrics=['accuracy'])
cnn_model.summary()
# Plot the model
plot_model(cnn_model, show_shapes=True, show_dtype=True)
cnn_model.save('Building_CNN.keras')
```

```
# Model Training and Validation
history = cnn_model.fit(X_t1, y_t1, batch_size=64, epochs=10, validation_data=(X_t2, y_t2))

# predict
pred_cnn = cnn_model.predict(X_t2)
y_pred_cnn = np.argmax(pred_cnn, axis=-1)

cnn_model.save('CNN_Model.keras')
```

```
# plotting graphs for accuracy
plt.figure(figsize=(10, 6))
plt.plot(history.history['accuracy'], label='training accuracy')
plt.plot(history.history['val_accuracy'], label='val accuracy')
plt.title('CNN Model Accuracy')
plt.xlabel('Epochs')
plt.ylabel('Accuracy')
plt.legend()
plt.show()

plt.figure(figsize=(10, 6))
plt.plot(history.history['loss'], label='training loss')
plt.plot(history.history['val_loss'], label='val loss')
plt.title('CNN Model Loss')
plt.xlabel('Epochs')
plt.ylabel('Loss')
plt.legend()
plt.show()
```

e) Model 5: K-Means Clustering

```
# Clustering the traffic signs using K-Means Clustering
from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import silhouette_score
from sklearn.decomposition import PCA

# Reshape X_t1 into a 2D array before applying StandardScaler
X_t1_reshaped = X_t1.reshape(X_t1.shape[0], -1)

# Standardize the data
scaler = StandardScaler()
X_normalized = scaler.fit_transform(X_t1_reshaped)

# Dimensionality Reduction for Visualization using PCA
# Reduce dimensions to 2D for visualization
pca = PCA(n_components=2)
X_train_pca = pca.fit_transform(X_normalized)

# Define the number of clusters for GTSRB
kmeans = KMeans(n_clusters=num_classes, random_state=42)
# Fit the K-Means model
kmeans.fit(X_train_pca)
# Assign clusters to images
clusters = kmeans.labels_
```

```python
# Visualize Clusters
plt.figure(figsize=(12, 8))
for i in range(num_classes):
    plt.scatter(X_train_pca[clusters == i, 0], X_train_pca[clusters == i, 1], label=f"Cluster {i}", alpha=0.6)

plt.title("K-Means Clustering of Traffic Signs Visualization with PCA")
plt.xlabel("Principal Component 1")
plt.ylabel("Principal Component 2")
plt.legend()
plt.show()

# Compute cluster quality
silhouette_avg = silhouette_score(X_train_pca, clusters)
print(f"Silhouette Score: {silhouette_avg:.2f}")
```

Step 4: Models Evaluation

```python
# Evaluation Metrics
from sklearn.metrics import confusion_matrix, accuracy_score, precision_score, recall_score, f1_score, classification_report

def evaluate_model(model_name, y_pred, y_test):
    y_test_labels = np.argmax(y_test, axis=1)
    accuracy = accuracy_score(y_test_labels, y_pred)
    precision = precision_score(y_test_labels, y_pred, average='macro')
    recall = recall_score(y_test_labels, y_pred, average='macro')
    f1 = f1_score(y_test_labels, y_pred, average='macro')
    classification_rep = classification_report(y_test_labels, y_pred)
    confmatrix = confusion_matrix(y_test_labels, y_pred)
    print(f"{model_name} - Accuracy: {accuracy:.2f}, Precision: {precision:.2f}, Recall: {recall:.2f}, F1 Score: {f1:.2f}")
    print(classification_rep)

    f,ax = plt.subplots(figsize=(30, 20))
    sns.heatmap(confmatrix, annot=True, linewidths=0.01, cmap="BuPu", linecolor="gray", fmt=".1f", ax=ax);
    plt.xlabel("Predicted Label");
    plt.ylabel("True Label");
    plt.title("Confusion Matrix");
    plt.show();
```

```python
# Evaluate Metrics for K-Means Clustering
from scipy.stats import mode

def compute_purity(y_true, y_pred, n_clusters):
    majority_sum = 0
    for cluster in range(n_clusters):
        indices = np.where(y_pred == cluster)[0]
        if len(indices) > 0:
            majority_class = mode(y_true[indices])[0][0]
            majority_sum += (y_true[indices] == majority_class).sum()
    return majority_sum / len(y_true)

purity = compute_purity(y_t1, clusters, num_classes)
print(f"Purity Score: {purity:.2f}")
```

```python
# Evaulating the models
print("Evaluation Metrics:")
# Evaluate the RF model
evaluate_model("Random Forest", y_pred_rf, y_t2)
# Evaluate the KNN model
evaluate_model("K-Nearest Neighbors", y_pred_knn, y_t2)
# Evaluate the CNN model
evaluate_model("CNN", y_pred_cnn, y_t2)
# Evaluate the TL model
evaluate_model("Transfer Learning", y_pred_mobilenet, y_t2)
tf.saved_model.save(cnn_model, 'ML_Models')
```

4.3 Experimental Results of ML Models

GTSRB is a widely used dataset in the automotive field for evaluating traffic sign classification models. The accuracy improvement typically required extensive data augmentation and careful fine tuning. The pre-trained features make it an ideal candidate for tasks like road sign classification, which may not require a large dataset for training when using transfer learning.

Model	Evaluation Metrics						Use Case
	Accuracy	Precision	Recall	F1-Score	Training Time	Speed	
Random Forest	0.98	0.99	0.98	0.98	Moderate	Fast	Small Datasets and quick prototypes.
K-Nearest Neighbors (KNN)	0.87	0.89	0.87	0.88	Moderate	Fast	Suitable for classification and regression tasks and no training required.
Transfer Learning	0.85	0.84	0.83	0.83	10 epochs	Fast	Leverage pre-trained models.
Convolutional Neural Network (CNN)	0.99	0.99	1.00`	0.99	10 epochs	Moderate	General purpose image classification.
K-Means Clustering	**Silhouette Score**: 0.34 (Measures the compactness of clusters). **Purity Score**: 0.42 (Compares clusters against true lables.)				Moderate	Fast	Useful for image segmentataion and efficient for large datasets.

Table 4: ML Models Experimental Results

Note: Measuring evaluation metrics like accuracy, precision, recall and F1-score for a K-means clustering model requires comparing the predicted cluster labels to the true labels of the dataset. However, since K-Means is an unsupervised learning algorithm, the predicted cluster labels are arbitrary. Without true labels, metrics like **silhouette score** and **purity score** can complement accuracy-based metrics to assess clustering quality.

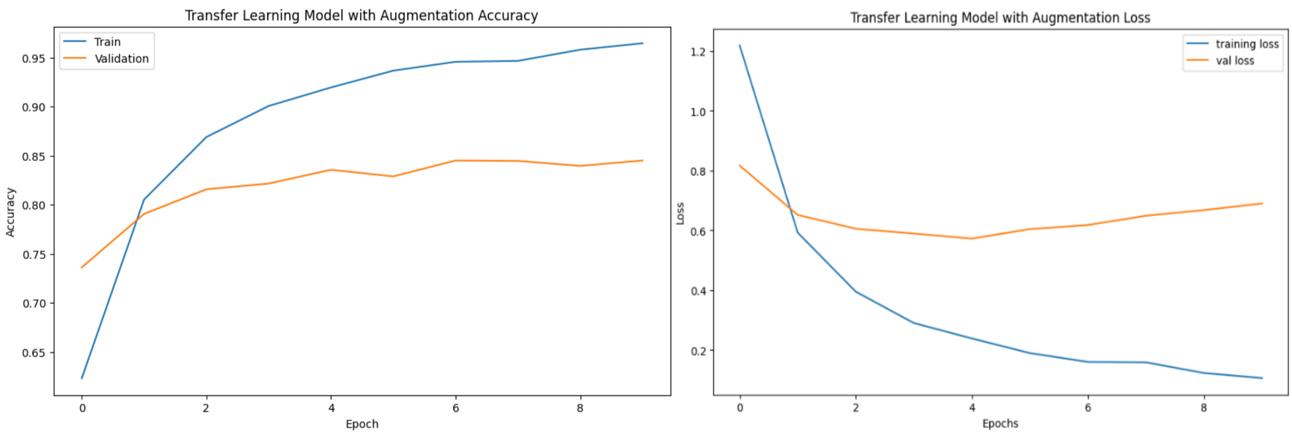
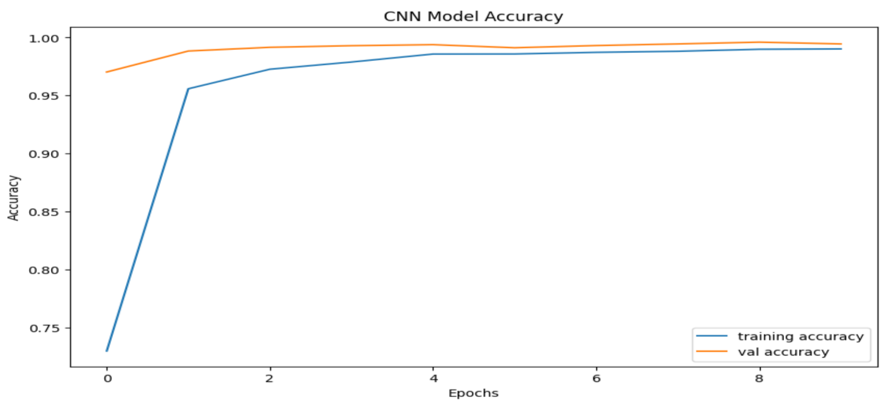


Figure 9: MobileNetV2 Transfer Learning Model Graphs



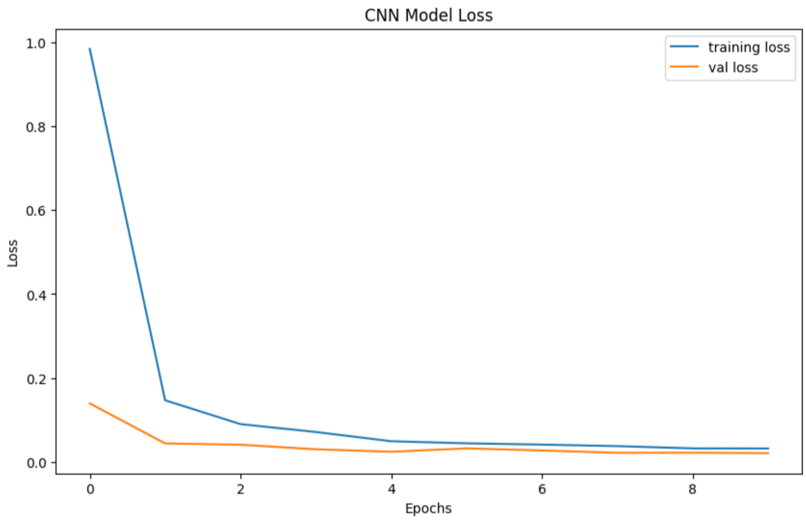


Figure 10: CNN Model Graphs

The below scatter plot showing the clustering of traffic signs in a 2D PCA-reduced space.

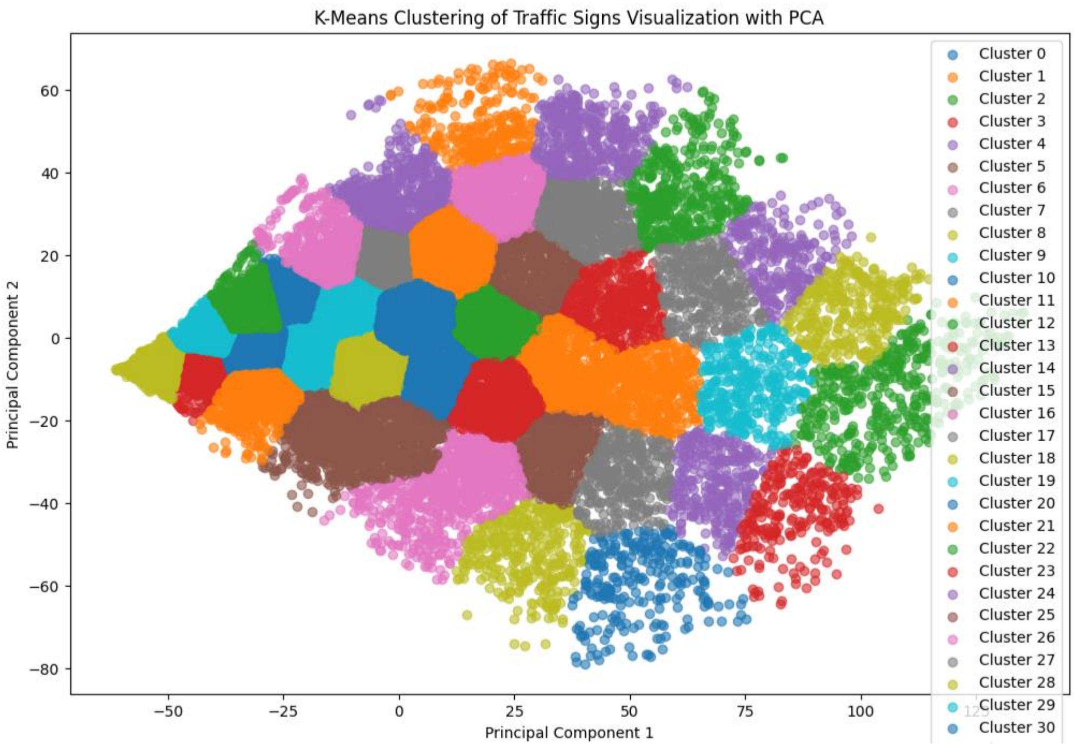


Figure 11: K-Means Clustering Visualization Diagram of GTSRB Dataset

After running the model's evaluation, we can see the below results

```
Evaluation Metrics:
Random Forest - Accuracy: 0.98, Precision: 0.99, Recall: 0.98, F1 Score: 0.98

K-Nearest Neighbors - Accuracy: 0.87, Precision: 0.89, Recall: 0.87, F1 Score: 0.88

CNN - Accuracy: 0.99, Precision: 0.99, Recall: 1.00, F1 Score: 0.99

Transfer Learning - Accuracy: 0.85, Precision: 0.84, Recall: 0.83, F1 Score: 0.83
```

K-Means Clustering - Purity Score: 0.42, Silhouette Score: 0.34

4.4 Key Insights from Published Results

1. Randon Forest and KNN are quick to implement but may underperform compared to deep learning models. Random Forest trains multiple decision trees and aggregate predictions. It is effective for moderately complex datasets.
2. K-Nearest Neighbors uses raw pixel data or extracted features. It is simple but computationally expensive for large datasets. Focus on data preprocessing, feature extraction and hyperparameter tuning to improve the accuracy.
3. CNN achieves high accuracy with relatively lightweight architecture. Uses interconnected layers of neurons to learn complex patterns in data.
4. MobileNetV2 with transfer learning achieves the best results due to pre-trained features. We achieved 85% accuracy in our current study with MobileNetV2 pre-trained model and it can match or outperform the CNN by fine tuning. MobileNetV2 expects input shapes in 224x224 weights, but we resized images to 64x64 to reduce computation latency & memory usage in our study. It means the network will have fewer spatial features to work with. And system crashes if uses above 64x64 image size.
5. K-Means Clustering quality depends on the dataset and feature representation. PCA is used here for 2D visualization and dimensionality reduction. Feature extraction (HOG, CNN features) may improve clustering quality compared to raw pixel values. We can also try t-SNE, DBSCAN and Agglomerative Clustering for better results in non-linear spaces.

4.5 Comparison of ML Models

Random Forest, KNN, CNN, Transfer Learning and K-Means Clustering are different popular approaches in Machine Learning, used for image classification and clustering. A comparison between them as shown below.

Metric	Random Forest	KNN	Trasfer Learning	CNN	K-Means Clustering
Accuracy	98%	87%	85&	99%	34%
Training Time	Moderate	Fast	10 epochs (shorter)	10 epochs (longer)	Moderate
Dataset	Perform well with small datasets	Requires a sufficient data, Moderate datasets	Perform well with smaller datasets	Require a large, labeled dataset	Require a large datasets
Performance	Excellent through optimized preprocessing and hyperparameter tuning.	Effective for smaller datasets.	Competitive with limited data, fine-tuning can match or outperform CNNs	Excellent, if trained with enough data	Good, if quality of feature extraction, number of clusters, and data preprocess
Model Complexity	Low risk of overfitting	Simple to implement but computationally expensive during inference.	Deeper, pre-trained network (higher)	Shallow Architecture (lower)	Low, works best with clean, well separated features.
Computational Cost	Moderate for GTSRB dataset.	High if uses full GTSRB dataset	Less to Moderate, requires only fine tuning.	High	Moderate for GTSRB
Generalization	Good due to its ensemble approach.	Limited, prone to underfitting and overfitting.	Excellent	Limited, prone to overfitting	Limited, dependent on centroid quality.
Strengths	Multi-class classification, less prone to overfitting and flexibility.	Simple and effective for smaller datasets and Non-parametric nature.	Pre-trained features, faster training, better accuracy, smaller datasets.	Customizable, smaller.	Simple to implement and fast on low-dimensional data.

Challenges	1. High Dimensional Data 2. Large Dataset Size 3. Fine-grained classes	1. High Dimensionality 2. Sensitive to noise, misclassified images affect predictions. 3. Computational complexity with large datasets like GTSRB.	1. Requires more memory, larger architecture. 2. Insufficient dataset size like GTSRB.	1. Require a large, labeled datasets. 2. Training from scratch requires large computational resources and time.	1. Unsupervised nature as doesn't use label information. 2. Feature Dependence 3. High Dimensionality

Table 5: Comparison of ML Models

Note: Verify Jupiter Notebook **Model Implementation** Source file in **Appendix**.

5 Conclusion

Machine Learning models play a critical role in handling and deriving insights from big data, especially in complex domains like autonomous vehicles. The combination of ML and big data enables systems to process vast amounts of information, recognize patterns, make predictions, and take real-time decisions[4].

The use of big data in conjunction with ML models for traffic sign recognition has shown great promise in enhancing safety, efficiency, and reliability of AV systems. By leveraging large-scale datasets and advanced ML models, such as those trained on the GTSRB dataset, significant progress can be made in the development of robust recognition systems for AVs.

The choice of ML model depends on the problem at hand, the type of data available, and the goal (classification, regression, clustering, etc.). Used four models from the supervised learning in the current study on the GTSRB dataset as these models trained on labeled data.

Randon Forest is an ensemble method that combines multiple decision tress to improve classification accuracy. By averaging the predictions of many trees, it reduces overfitting. GTSRB contains images with multiple features, such as color, shape, and texture. Randon Forest can effectively handle this high dimensional data, making it suitable for image classification tasks. Training on the GTSRB dataset involved parameters tuning (number of trees, max.depth) to prevent underfitting and overfitting. The model performance improves especially when the number of trees is sufficiently large.

KNN is a non-parametric. Instance-based learning algorithm that classifies a data point based on the majority label of its ‘k’ nearest neighbors in the feature space. On the GTSRB dataset, KNN can achieve good performance, especially when the ‘k’ value is properly tuned. A small ‘k’ can make the model sensitive to noise, while a large ‘k’ can smooth out the decision boundaries and make the model less responsive to local patterns.

CNN from Scratch is effective but often requires heavy data augmentation and fine-tuning to perform well. While it is flexible, the need for a large dataset and longer training time makes it less favourable for tasks like road sign classification in practical applications.

Transfer Learning (MobileNetV2) provides significant advantages in both performance and training time. It reaches higher accuracy with much less training effort due to the powerful pre-trained feature extraction layers. This is why it has become the go-to approach in automotive classification tasks like road sign recognition.

K-Means clustering algorithm is a popular unsupervised learning method, applied on GTSRB dataset for tasks like traffic sign categorization and feature extraction. While K-Means is not inherently a classification algorithm, it has potential in clustering tasks related to traffic sign recognition. Used K-Means for grouping traffic-sign images into clusters based on their visual similarities and is useful for discovering inherent patterns in the dataset without requiring labeled data. It also uses PCA for dimensionality reduction by identifying cluster centroids, which can then represent traffic-sign images in a lower dimensional space. It can also use for data preprocessing for other machine learning models.

Hadoop and Spark also use for scaling and parallelizing computations in automotive applications like traffic sign recognition, autonomous driving tasks. However, their usage is generally more focused on large-scale data processing rather than deep learning tasks like training CNNs. Hadoop and Apache Spark are widely recognized for their ability to handle large-scale data processing in a distributed environment, particularly for preprocessing, feature extraction, and big data analysis in automotive and autonomous driving research.

Appendix: ML Models Source Code

ML Models Implementation Source Code File.

GTSRB_BigData_ML_Models_v1_0.ipynb